

How Shield VPN Works

A deep dive — diagrams, analogies, networking internals & interview prep

Everything behind the app we built: how a phone tap turns into an encrypted tunnel exiting from a server in Sweden, how the packets actually move, why WireGuard/OpenVPN/custom differ, how to scale to many servers — plus interview questions with answers.

Networking internals

WireGuard

OpenVPN

Custom / DPI

Scaling

Interview Q&A

Project: Shield VPN (Flutter + WireGuard on AWS) • **Server:** AWS EC2, Stockholm • **Audience:** engineers & interview prep

1. The big picture (in one analogy)

Analogy — the sealed diplomatic pouch. Normally when you send mail, every post office along the way can read the envelope: who it's from, who it's to, and (postcards) the message. A VPN is like handing your letter to a **trusted courier** who seals it in an opaque, tamper-proof pouch, drives it to an **embassy in another country**, and only there is it opened and sent onward. Anyone watching the road sees a pouch going to the embassy — not your letter, not its destination. The reply comes back the same way.

In that analogy:

Analogy	Real VPN thing
Your letter	Your data packets (the website you visit, the request)
The sealed pouch	Encryption (ChaCha20 / AES) — nobody on the path can read inside
The courier & route	The VPN tunnel over the internet (UDP port 51820)
The embassy abroad	The VPN server (your AWS box in Sweden)
“From the embassy” postmark	The server’s IP — the website thinks <i>the server</i> is the visitor, not you
People watching the road	Your ISP, Wi-Fi owner, network snoopers

The one sentence: A VPN wraps your traffic in encryption and routes it through a server elsewhere, so (a) the local network can’t read it, and (b) the destination sees the *server’s* identity instead of yours.

2. What we actually built (the journey)

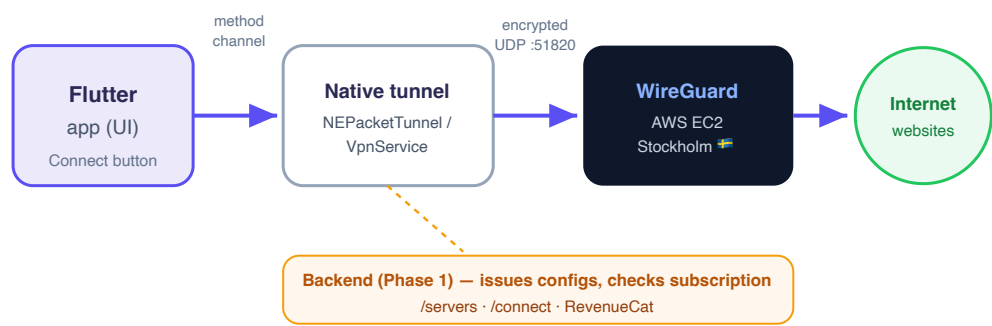


Figure 1 — the four pieces. Dart can’t tunnel traffic; the native extension does. The tunnel is encrypted; only the server can open it.

The exact steps we did

- 1. **Rented a server** — an AWS EC2 Ubuntu box in Stockholm; opened UDP port 51820 in the security group; disabled source/dest check so it can route.

2. **Installed WireGuard** — generated a server key-pair and a client key-pair; wrote `wg0.conf`; enabled IP forwarding + NAT (masquerade).
3. **Proved it** — pasted the client config into the desktop WireGuard app; our public IP changed to `51.21.255.243` (Sweden). The tunnel works.
4. **Built the Flutter app** — Home/Servers/Speed UI, a `VpnEngine` abstraction (mock now, real WireGuard later), real IP + real speed test.

3. The networking side (deep dive)

3.1 How the internet normally works

Your data is chopped into **packets**. Each packet has a header (like an envelope) with a **source IP** and **destination IP**, and a payload (the contents). Routers along the way read the destination IP and forward the packet hop-by-hop until it reaches the server; the reply comes back to your source IP.

Analogy — postcards through many post offices. A normal request is a stack of postcards. Every router (post office) reads the “to” and “from” addresses to pass it along, and can glance at the message. Your ISP is the first post office and sees *every* address you write to.

3.2 What the VPN changes: the tunnel

A VPN puts each of your real packets **inside another packet** (encapsulation) and encrypts the inside. This is a **tunnel**: an outer packet whose only visible addresses are *you* → *the VPN server*. The real destination is hidden inside the encrypted payload.

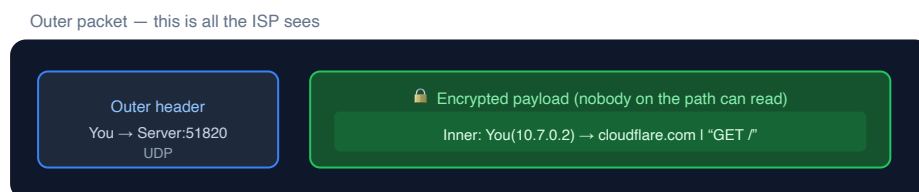


Figure 2 — encapsulation. The real request is sealed inside; the outside only shows “me talking to my VPN server.”

3.3 The TUN interface — where packets are grabbed

The OS gives the VPN a virtual network card called a **TUN device**. Your phone’s routing table is changed so that `AllowedIPs = 0.0.0.0/0` (“all traffic”) goes into this virtual card instead of out the Wi-Fi. The VPN process reads raw packets from TUN, encrypts them, and sends them to the server over UDP. Replies are decrypted and written back into TUN, so apps think they came from the real internet.

Analogy — a mail chute that reroutes everything. TUN is a chute installed in front of the normal mailbox. Every outgoing letter now drops into the chute (the VPN), gets sealed and re-addressed to the embassy, and only then goes out. Apps never know — they still “drop mail” the same way.

3.4 NAT / “masquerade” — how the server forwards you

Inside the tunnel you have a private address like `10.7.0.2`. That address means nothing on the public internet. So the server rewrites the source of your outgoing packets to **its own public IP** — this is **NAT (Network Address Translation)**, the `MASQUERADE` rule we added. It remembers the mapping so replies come back to you.

Analogy — the hotel front desk. You (room 10.7.0.2) ask the front desk to order takeaway. The desk calls the restaurant using the *hotel's* phone number and address. The restaurant only ever knows the hotel. When food arrives, the desk checks its notebook and sends it to your room. That notebook is the NAT table.

3.5 Ports, UDP vs TCP

	UDP (WireGuard uses this)	TCP (OpenVPN can use this)
Style	“Fire and forget” — fast, no handshake per packet	Reliable, ordered, acknowledged
Speed	Faster, lower latency	Slower (more overhead)
Firewall evasion	Easier to block (unusual port)	On port 443 it looks like HTTPS → hard to block
Our port	<code>51820/udp</code>	<code>443/tcp</code> for stealth

3.6 Keys & the handshake (public-key crypto)

Each side has a **key-pair**: a **public key** (shareable) and a **private key** (secret). WireGuard uses them to agree on a temporary shared secret and to verify identity — the **Noise handshake**. After that, data is encrypted with fast symmetric crypto (ChaCha20-Poly1305).

Analogy — the padlock and key. Your public key is an *open padlock* you hand out freely. Anyone can snap it shut on a box, but only your *private key* opens it. Both sides swap padlocks, so each can send boxes only the other can open — and each proves who they are by opening the right box.

3.7 DNS — the address book (and leaks)

Before connecting to `cloudflare.com`, your device asks a **DNS resolver** “what’s the IP?”. If those DNS questions leak *outside* the tunnel, your ISP still sees every site name you visit — a **DNS leak**. That’s why our client config sets `DNS = 1.1.1.1` and routes it *through* the tunnel.

Analogy — asking directions. Even inside the sealed pouch, if you roll down the window to ask a passer-by “where’s cloudflare.com?”, you’ve revealed where you’re going. DNS-through-the-tunnel means you only ask *inside* the embassy.

4. WireGuard — how our tunnel works

WireGuard is deliberately tiny (~4,000 lines) with fixed modern crypto. Its core idea is **cryptokey routing**: a peer is identified by its public key, and each public key is tied to the IPs it’s allowed to use (`AllowedIPs`).

```

[Interface]                # ← this device (the client)
PrivateKey = <client_secret>
Address      = 10.7.0.2/32   # my address inside the tunnel
DNS          = 1.1.1.1

[Peer]                     # ← the server
PublicKey    = <server_public>
Endpoint     = 51.21.255.243:51820
AllowedIPs   = 0.0.0.0/0, ::/0 # send ALL my traffic through this peer
PersistentKeepalive = 25

```

- **AllowedIPs** does double duty: outbound it's a route ("which traffic goes to this peer"); inbound it's an access rule ("which source IPs I accept from this peer").
- **Stateless & roaming:** WireGuard doesn't care if your phone's IP changes (Wi-Fi→4G); it re-binds automatically. Great for mobile battery + reconnection.
- **Server side** keeps a list of peers (one per user/device). Adding a user = adding a **[Peer]** with their public key and a unique **10.7.0.x**.

5. OpenVPN — the older, stealthier cousin

OpenVPN predates WireGuard and works differently: it builds a **TLS session** (the same technology as HTTPS) and tunnels your packets inside it. It uses **X.509 certificates** (a Certificate Authority signs each client) rather than raw key-pairs.

Aspect	WireGuard	OpenVPN
Transport	UDP only	UDP <i>or</i> TCP
Crypto	Fixed (Curve25519 + ChaCha20)	Configurable (OpenSSL, AES-256-GCM...)
Identity	Public keys	Certificates (CA / PKI)
Speed / battery	★★★★★	★★★
Looks like HTTPS?	No	Yes, on TCP/443 → beats many firewalls
Config size	Tiny	Large (embeds CA + cert + key)

Analogy. WireGuard is a **motorbike courier** — lean, fast, unmistakable. OpenVPN is a **plain grey delivery van that looks like every other van on the road** (it looks like normal HTTPS web traffic), so censors struggle to single it out — at the cost of being heavier and slower.

Product strategy: ship WireGuard by default; auto-fall back to OpenVPN-TCP/443 when a network blocks UDP.

6. Custom / obfuscated protocols — beating DPI

Some networks use **Deep Packet Inspection (DPI)**: they don't just read addresses, they fingerprint the *shape* of traffic and block "this looks like a VPN." To defeat that you **obfuscate** — disguise the tunnel as ordinary traffic.

Analogy — a disguise, not a new lock. You don't invent a new pouch material (never roll your own crypto). Instead you make the courier **look like an ordinary pizza delivery** so the guards wave it through. The contents are still sealed by proven encryption.

Three honest levels of “custom”

1. **Obfuscation wrapper (recommended)** — keep WireGuard/OpenVPN doing the crypto, wrap it to look like normal TLS/WebSocket: `AmneziaWG`, `wstunnel`, `XRay/VLESS`, `obfs4`.
2. **Custom transport, vetted crypto** — your own packet format + port-hopping, but built on the **Noise framework** / `libsodium`. Ship one Go core to both platforms via `gomobile`.
3. **Fully bespoke protocol** — only with a security team + external audits. Not a starting point.

Rule #1 of VPN engineering: never invent your own encryption. “Custom” means a new *disguise* around proven crypto, not new crypto.

7. What if we add more servers?

One server = one exit country and a bandwidth ceiling. A real product has many. Here's what changes.

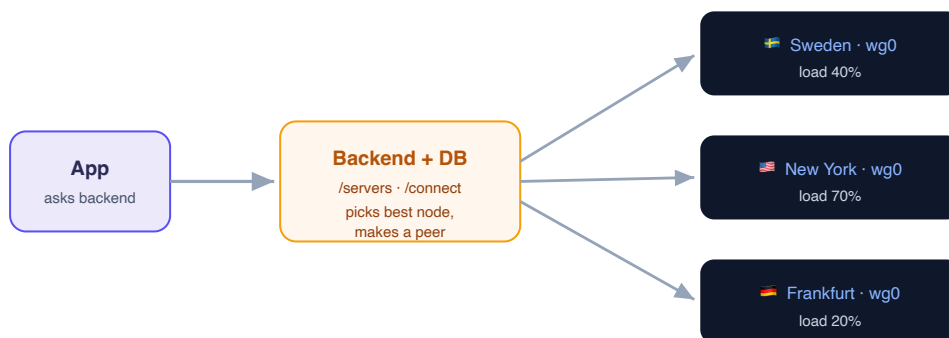


Figure 3 — with many servers, a backend becomes essential: it stores the fleet, picks a node, and creates the peer on demand.

What has to exist once you have >1 server

- **A server list (DB + `GET /servers`)** — id, country, city, host, current load, protocols. The app shows this list (exactly our Servers screen).
- **Dynamic peer creation (`POST /connect`)** — the backend verifies the subscription, generates a fresh key-pair, runs `wg set wg0 peer ...` on the chosen server (via `wgctrl` or an agent), and returns the config. On disconnect it removes the peer.
- **IP address management** — each peer needs a unique `10.7.0.x` per server; the backend tracks the pool.
- **Load balancing / “fastest server”** — pick by lowest load or lowest latency to the user (like our ping values).
- **Provisioning automation** — Terraform + Ansible so “add Tokyo” is one command: boot VM, install WireGuard, set NAT, register in the DB. Tools: **wg-easy**, **Netmaker**, **Firezone**.
- **Monitoring & health** — drop unhealthy nodes from `/servers` automatically.

- **Bandwidth cost** — this is the real bill. AWS egress (~\$0.09/GB) is far too expensive at scale; production VPNs use flat-rate/unmetered hosts.

Analogy — one embassy vs. a global network. One embassy handles a trickle of mail. A worldwide courier company needs a *dispatch office* (the backend) that knows every branch, how busy each is, and routes each customer to the nearest, least-crowded one — issuing them a one-time access badge (the peer config) as they go.

8. Interview questions & answers

Networking fundamentals

Q What's the difference between TCP and UDP, and why does WireGuard use UDP?

A TCP is connection-oriented, reliable, ordered, and acknowledges every segment; UDP is connectionless and fire-and-forget. WireGuard uses UDP for low latency and because a VPN carries TCP *inside* it already — putting TCP-in-TCP causes “meltdown” (two retransmit timers fighting). UDP avoids that and roams better across network changes.

Q What is NAT and where does it happen in a VPN?

A NAT rewrites the source (or destination) IP/port of packets and remembers the mapping so replies return correctly. In our VPN the server does source-NAT (MASQUERADE): it replaces the client's private `10.7.0.x` with the server's public IP on the way out, and reverses it on replies. That's why the website sees the server, not you.

Q What is a DNS leak and how do you prevent it?

A When DNS queries go outside the tunnel (e.g. to the ISP's resolver), an observer still learns which sites you visit even though the traffic is encrypted. Prevent it by pushing a DNS server in the client config and routing DNS through the tunnel, and by blocking IPv6/DNS that bypasses the interface.

Q Walk me through what happens, packet by packet, when I tap “Connect” and open a website.

A (1) The app starts the tunnel; a handshake with the server derives session keys. (2) The OS route table sends all traffic to the TUN device. (3) A web request packet `10.7.0.2 → cloudflare` is read from TUN, encrypted, wrapped in a UDP packet `me → server:51820`, and sent. (4) The server decrypts it, NATs the source to its own IP, forwards it to Cloudflare. (5) The reply returns to the server, which encrypts it and sends it back; the client decrypts and writes it into TUN, and the browser receives it as if normal.

Crypto & security

Q Explain public vs private keys in this system.

A Each peer has a key-pair. The public key is shared and identifies the peer (cryptokey routing); the private key never leaves the device. WireGuard combines them in a Diffie-Hellman/Noise handshake to derive a shared symmetric key used for fast per-packet encryption (ChaCha20-Poly1305), with forward secrecy via periodic re-keying.

Q Why should you never write your own encryption for a “custom” protocol?

A Cryptography is extraordinarily easy to get subtly wrong (nonce reuse, padding oracles, timing side-channels), and bugs are invisible until exploited. Production “custom” protocols reuse vetted primitives (Noise, libsodium, BoringSSL) and only customise the *transport/obfuscation* layer, which is then independently audited.

Architecture & system design

Q Why can’t the Flutter (Dart) code do the tunnelling itself?

A Operating systems only grant packet-capture/routing to a sandboxed system network extension — `NEPacketTunnelProvider` on iOS, `VpnService` on Android. Dart runs in the app sandbox and can’t touch the TUN device, so the tunnel must live in native code; Flutter drives it over a method channel.

Q Design the backend for a VPN with millions of users and hundreds of servers.

A Stateless API behind a load balancer; Postgres for users/subscriptions/server inventory; Redis for sessions/rate-limits. `GET /servers` returns healthy nodes with live load. `POST /connect` checks entitlement, picks the best node (geo + load), creates a short-lived peer via `wgctrl` /agent, returns the config, and schedules cleanup on disconnect/expiry. Provision nodes with Terraform/Ansible; health-check and auto-evict. Bill and cap by bandwidth; keep a no-logs policy at the data layer.

Q How would you handle a network that blocks WireGuard?

A Detect the failure (no handshake), then fall back through a protocol ladder: WireGuard-UDP → OpenVPN-TCP/443 (looks like HTTPS) → an obfuscated transport (AmneziaWG / XRay) that disguises the tunnel against DPI. Expose this as an “auto / stealth” toggle in the app.

Q What are the main scaling bottlenecks?

A Bandwidth cost (dominant — pick unmetered hosts, cap free tier), per-server connection/CPU limits (crypto throughput), IP pool management, control-plane latency when creating peers, and abuse handling (exit nodes attract DMCA/fraud complaints, which can get servers deprovisioned by the host).

Product / business

Q How do free VPNs afford servers?

A Either they monetise you (selling browsing data or reselling your bandwidth as a residential proxy — the shady models), or clean **freemium**: a hard-capped free tier subsidised by a small % of paying users. A trustworthy product uses subscriptions (RevenueCat) with a genuine no-logs stance, and caps free bandwidth so the AWS bill doesn’t explode.